NanoParse

Cloud Computing Assignment Report

A lightweight natural-language command parser using knowledge distillation, containerised inference, S3-compatible model storage and Kubernetes orchestration.

Project	NanoParse
Student	Rahul Singh Jadoun
Repository	github.com/rahul-singh92/NanoParse
Production model	Pythia-70M fine-tuned student — v1
Experimental model	v2 — not selected for production

Prepared from project configuration, source files and recorded experiment evidence.

1. Overview and Architecture

NanoParse converts short natural-language commands into a fixed JSON frame containing an intent and optional date, time and place fields. A larger teacher model generates examples; the data is cleaned and used to fine-tune a compact Pythia-70M student model. FastAPI serves the selected model as an API.

Request path: Client command → FastAPI /parse → Pythia-70M inference → JSON-frame validation → structured response. Model artifacts are stored separately in SeaweedFS, an S3-compatible object store. Docker packages the API; k3d provides a local Kubernetes cluster.

Layer	Technology
Teacher / data generation	Ollama Phi-3.5
Student / ML	EleutherAI Pythia-70M, PyTorch, Transformers
API	FastAPI and Uvicorn
Storage	SeaweedFS S3-compatible object storage
Orchestration	Docker, Kubernetes, k3d, Helm
Testing / automation	unittest, Ruff, GitHub Actions, Gitleaks, Locust

2. Data Preparation and Model Evaluation

The teacher-generated command dataset contained 3,000 examples. Cleaning checked JSON syntax, required keys, valid intents and slots, and duplicates. The v1 cleaned training set retained 2,292 examples. Recorded exclusions: 1 invalid JSON output, 57 wrong-key examples, 610 invalid-slot examples and 40 duplicates. The cleaning categories may overlap; the retained count is the definitive output count.

Validation metric (100 examples)	Teacher Phi-3.5	Student v1	Student v2
JSON validity	94%	90%	89%
Intent accuracy	85%	75%	76%
Date accuracy	71%	68%	69%
Time accuracy	79%	76%	79%
Place accuracy	81%	77%	71%
Full-frame accuracy	44%	41%	42%

V1 remains the production model. V2 raised full-frame accuracy by one percentage point and improved intent, date and time slightly, but reduced JSON validity and place accuracy. Neither student reaches the teacher baseline on the reported validation metrics. baseline_validation.json contains zero scores and is treated as an untrained baseline, not a trained-model result.

3. Containerised API and Deployment

The FastAPI service exposes GET /healthz, GET /readyz and POST /parse. At startup it downloads the selected model archive from S3-compatible storage, extracts it with a path-traversal guard, and loads it on CPU. Readiness remains unavailable until model loading completes. Successful parsing responses include X-Model-Version and X-Served-By headers; invalid model output returns HTTP 500.

The container uses Python 3.12 slim and pinned inference dependencies, including CPU-only PyTorch. The recorded ARM64 image size was approximately 1.66 GB, within the 2 GB target. A resource test with CPU and memory limits recorded peak cgroup memory of approximately 737.4 MiB and no OOM kill.

Kubernetes component	Configuration / role
Deployment	nanoparse API; two replicas during Task 7
Service / Ingress	Service exposure and Traefik path routing for /parse
HPA	CPU-based scaling; minimum 1, maximum 2; target 70% CPU
Update strategy	Rolling update; maxSurge 1, maxUnavailable 0
Storage	SeaweedFS S3-compatible service in namespace data

The deployment script creates the k3d cluster when absent, installs SeaweedFS with Helm, downloads and verifies the selected model release, uploads the archive, builds and imports the service image, applies Kubernetes manifests and waits for rollout. Model archives are distributed through the GitHub Release rather than committed to Git.

Deployment evidence recorded successful rollout, model-version response headers, readiness changing from 503 during model loading to 200 when ready, and rollback from v2 to v1. Cold-start time from pod creation to Ready was 11 seconds in each of three recorded runs.

4. Networking and Access Controls

The network design divides the 10.20.0.0/16 address space into four subnets. The ACL simulator applies stateless, first-match rules and checks sample packet decisions.

Subnet	CIDR	Usable addresses	Capacity
Application	10.20.0.0/22	10.20.0.1–10.20.3.254	1,022
Public	10.20.4.0/24	10.20.4.1–10.20.4.254	254
Data	10.20.5.0/25	10.20.5.1–10.20.5.126	126
Management	10.20.5.128/27	10.20.5.129–10.20.5.158	30

Inbound rules deny the specified blocked source range first, allow HTTP and HTTPS, restrict SSH to the management subnet, and allow ephemeral return traffic from the application subnet. Outbound rules allow application traffic to the application subnet and ephemeral return traffic; unmatched traffic is denied. The recorded simulator check passed 6 of 6 sample packets (100%).

5. CI, Testing and Security

GitHub Actions runs on pushes and pull requests to main, and supports manual dispatch with a trained or untrained model source. Stages: Python compilation and Ruff lint checks, unit tests, model evaluation, and Gitleaks secret scanning. Evaluation downloads the v1 release and verifies its SHA-256 checksum before generating validation predictions.

Normal push/PR evaluation reports whether JSON validity meets 95%, but does not fail solely on that threshold. Manual evaluation enforces the 95% threshold. A recorded successful CI run completed lint, five unit tests, evaluation and secret scanning. A separate manual untrained-model run failed the quality gate as expected; Gitleaks reported no leaks.

The trained v1 model's measured JSON validity is 90%, so it does not meet the 95% quality gate. This limitation is reported explicitly.

6. Task 7: Load Test, Resources, Cost and SLO

The recorded Locust run used four concurrent users, a spawn rate of four users per second and a two-minute duration against http://localhost:8080/parse. It produced 89 requests: 78 succeeded and 11 failed. Successful response headers show both replicas served traffic: 41 from one pod and 37 from the other. All successful responses reported model version v1.

Load-test measure	Observed result
Requests / successes / failures	89 / 78 / 11
Error rate	12.36%
Median response time	3,700 ms
Average response time	5,297 ms
p95 response time	16,000 ms
Minimum / maximum	1,100 ms / 30,013 ms
Throughput	0.748 requests/sec
Failure breakdown	9 HTTP 500; 2 HTTP 0 / no response

The error rate is 12.36%, and p95 latency is 16 seconds. Both miss the assignment targets of at most 1% errors and at most 5 seconds p95. The run demonstrates that two replicas served traffic, but the service did not meet the performance objectives.

Resource utilisation

Each pod had limits of 1 CPU and 1 GiB memory. Highest sampled CPU readings were approximately 993m and 989m; highest sampled memory was approximately 556 MiB and 555 MiB. CPU was nearly saturated; memory remained around 54% of the limit. These are peak sampled values, not averages.

Illustrative cost estimate

The exercise rate card is hypothetical: ■3.50 per vCPU-hour and ■0.45 per GiB-memory-hour. At one vCPU and one GiB per replica, two replicas cost ■7.90/hour. Using the measured rate (0.748439 requests/sec), the estimate is approximately ■2.93 per 1,000 attempted requests. This is not an actual cloud-provider bill.

SLO and error budget

For a 99% successful-request SLO over this run, the nominal error budget is 1% of 89 requests, or 0.89 failures (approximately one request). The 11 observed failures consume approximately 1,236% of this fractional budget. The SLO is not met. A two-minute test is a limited sample, not a long-term SLO measurement.

7. Review, Limitations and Conclusion

Area	Evidence / limitation	Improvement
Model quality	V1: 90% JSON validity and 41% full-frame validation accuracy.	Improve data coverage and output reliability; retain validation and quality gating.
Latency / throughput	p95 16 s; 0.748 req/s; 12.36% errors; CPU peaks near one core.	Profile inference, reduce CPU bottlenecks and retest at staged concurrency.
Resources	CPU nearly saturated; memory peak about 556 MiB per pod.	Benchmark resource requests/limits under longer tests.
Storage	Local SeaweedFS S3; model checksum verified.	Use replicated persistent storage, backups and production-grade access controls.
Reliability	Two replicas served successes, but 11/89 requests failed; single-node cluster.	Test multi-node recovery and improve malformed-output handling.
Security / cost	Gitleaks scan found no secrets; cost uses hypothetical rates.	Use managed secrets, TLS, least privilege and actual billing data.

Known limitations

The compact model has limited validation quality. The load test exposed high tail latency and failures. The Kubernetes environment is a local, single-node k3d cluster, so it does not establish multi-node or cloud-region resilience. The cost figure is illustrative only. A clean Gitleaks scan is evidence from the recorded scan, not a complete security certification.

Conclusion

NanoParse demonstrates an end-to-end ML application workflow: teacher-generated data preparation, student fine-tuning, model artifact storage, containerised inference, Kubernetes deployment, autoscaling, network ACL simulation, CI evaluation and load testing. V1 remains the production candidate because v2 did not provide a consistent validation improvement. The measured gaps are clear: v1 misses the 95% JSON-validity gate, and Task 7 misses both latency and error-rate targets. These are the next engineering priorities.

Evidence files

Key repository evidence: results/student_validation.json, results/teacher_validation.json, results/v2_score.json, results/task3-resource-summary.txt, results/task4/, results/task5-nacl.log, results/task6/, results/task7/, results/task7-resource-usage.log and results/loadtest_headers.json.

End of report